

System Design Document

GraduPredict Graduate Placement Prediction System

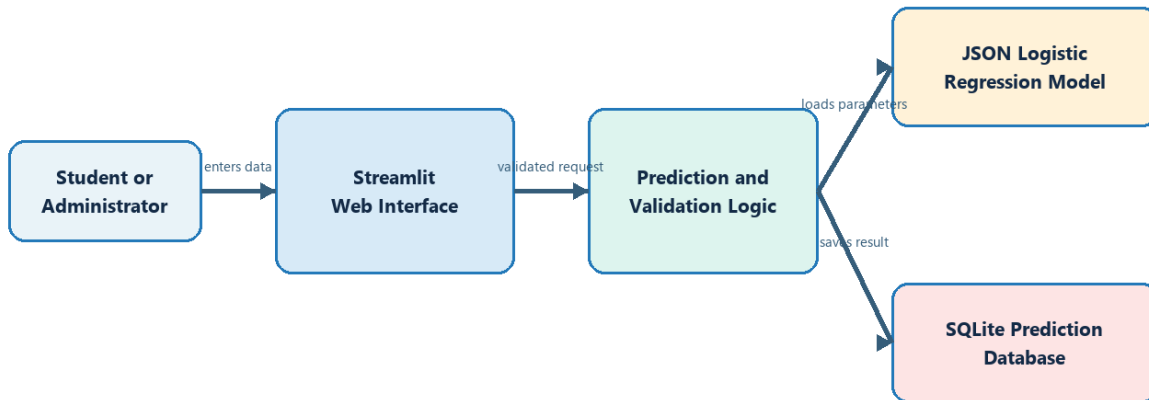
BS Computer Science Final Year Project
Version 1.0 | October 2026

Design Overview

GraduPredict uses a small layered design that one student can implement, explain, test, and deploy. The interface gathers input, the prediction module applies stored Logistic Regression parameters, and SQLite records only the final result and essential identifiers.

1 Architecture

GraduPredict System Architecture



Layer	Files	Responsibility
Presentation	app.py and Streamlit configuration	Navigation, forms, messages, metrics, tables, and downloads.
Prediction	prediction.py	Model loading, standardization, probability calculation, and classification.
Training	train_model.py and notebook.ipynb	Dataset loading, reproducible split, fitting, evaluation, and JSON export.
Persistence	SQLite database	Prediction ID, student ID, outcome, probability, risk, and timestamp.
Configuration	requirements.txt and placement_model.json	Runtime dependencies, feature order, scaling values, weights, threshold, and metrics.

2 Component Design

2.1 Prediction Pipeline

The training pipeline loads ordered features, performs an 80 percent training and 20 percent test split, calculates scaling statistics from the training partition, fits binary Logistic Regression by gradient descent, evaluates held-out predictions, and writes parameters to JSON. Runtime prediction applies the same feature order, mean, and scale before the sigmoid calculation.

2.2 Interface Flow

Step	User action	System response
1	Open application	Dashboard and navigation appear.
2	Open New Prediction	Validated student form appears.
3	Enter values and submit	System calculates probability, outcome, and risk.
4	Review and save	System inserts the result into SQLite.
5	Open history	System shows, filters, and exports records.
6	Open About Model	System explains the algorithm, metrics, dataset note, and limitation.

3 Database Design

GraduPredict Database Design

PREDICTIONS id INTEGER Primary Key student_id TEXT Required prediction TEXT Required probability REAL Required risk TEXT Required created_at TEXT Default timestamp
--

A single table is sufficient because the project does not manage user accounts or long-term student profiles. This reduces privacy exposure and keeps the scope appropriate for one student.

4 Detailed Prediction Design

Element	Design
Input vector	Eleven numeric values in the exact feature order stored in JSON.
Preprocessing	Subtract training mean and divide by training standard deviation.
Score	Weighted sum of standardized features plus intercept.
Probability	Sigmoid of the score.
Outcome	Likely placed at or above the selected threshold; otherwise At risk.
Risk	Low at 0.70 or above, Medium from 0.45 to below 0.70, and High below 0.45.
Storage format	Plain JSON to avoid scikit-learn pickle compatibility failures.

5 Security Privacy and Ethics

The form uses a project student identifier and excludes names, phone numbers, addresses, photographs, gender, ethnicity, religion, and disability. The result is an academic counseling estimate. The interface states that it must not determine hiring eligibility. Deployment should restrict database access and avoid publishing real student records.

6 Error Handling

Condition	Response
Blank student ID	Display an error and do not save.
Out-of-range value	Prevent entry through bounded controls.
Missing database	Create the table automatically.
Missing or malformed model file	Stop with a visible application error and restore or retrain the model.
No history records	Show an empty table and zero dashboard metrics.

7 Testing Design

Test level	Coverage
Unit	Probability range and classification boundaries.
Integration	Form to prediction module and save to SQLite.
System	Dashboard, entry, result, history, export, and model-information workflow.
Usability	Single user completes the viva flow with no assistance.
Data	Required columns, numeric ranges, target values, duplicates, and missing values.

8 Deployment Design

Local deployment uses a virtual environment followed by installation from requirements.txt and the command streamlit run app.py. Streamlit Community Cloud deployment uses a GitHub repository containing app.py, prediction.py, requirements.txt, model JSON, database directory, and configuration. Real student records should not be committed to a public repository.

9 Project Structure

The root contains application, prediction, training, requirements, README, and notebook files. Data, model, database, tests, documents, diagrams, and screenshots use separate named folders. This structure maps directly to the implementation responsibilities and simplifies viva explanation.